



Facultad de Ingeniería
Escuela de Ingeniería Informática

Bioinspirados con conciencia; un enfoque de explicabilidad para anticipar el estancamiento

Maximiliano Alejandro Aguirre Faúndes

Trabajo realizado para optar al título de
Ingenier(o/a) Civil Informático(o/a)

Prof. Guía: **Rodrigo Olivares Órdenes**

Valparaíso, Mayo 2026

Certificación

Certifico que he leído este documento y que, en mi opinión, es adecuado en ámbito y calidad como trabajo para optar al título de Ingenier(o/a) Civil Informático(o/a).

Rodrigo Olivares Órdenes
Profesor Guía

Certifico que he leído este documento y que, en mi opinión, es adecuado en ámbito y calidad como trabajo para optar al título de Ingenier(o/a) Civil Informático(o/a).

Nombre (del/de la) profesor(a)
Profesor Informante

Certifico que he leído este documento y que, en mi opinión, es adecuado en ámbito y calidad como trabajo para optar al título de Ingenier(o/a) Civil Informático(o/a).

Nombre (del/de la) profesor(a)
Profesor Informante

Aprobado por la Escuela de Ingeniería Informática, Universidad de Valparaíso.

Declaración sobre el uso de herramientas de Inteligencia Artificial

Yo, **Maximiliano Alejandro Aguirre Faúndes**, autor del trabajo “*Bioinspirados con conciencia; un enfoque de explicabilidad para anticipar el estancamiento*”, declaro que el contenido de este documento es original y ha sido elaborado principalmente mediante mi propio esfuerzo intelectual, en conformidad con los principios de integridad académica exigidos por la universidad.

Declaro asimismo que se han utilizado herramientas de Inteligencia Artificial, tales como Gemini, ChatGPT y Claude, exclusivamente como apoyo en tareas auxiliares de redacción, mejora de estilo, corrección gramatical y correcciones técnicas de implementación. Estas herramientas no han sido empleadas para la generación autónoma de contribuciones sustantivas del trabajo, tales como el planteamiento del problema, el diseño metodológico, la implementación ni el análisis de resultados, los cuales corresponden íntegramente al autor.

Todo contenido asistido por IA ha sido revisado, validado y adaptado críticamente, garantizando su pertinencia, exactitud y originalidad. Asumo plena responsabilidad por el contenido final del documento.

Se deja constancia de que el uso de estas herramientas ha sido realizado con conocimiento de **Rodrgio Olivares Órdenes**, profesor/a guía, en conformidad con las normativas institucionales y buenas prácticas académicas vigentes.

Bioinspirados con conciencia; un enfoque de explicabilidad para anticipar el estancamiento

Maximiliano Alejandro Aguirre Faúndes^{a,*}, Rodrigo Olivares Órdenes^a

^a*Escuela de Ingeniería Informática, Universidad de Valparaíso, Valparaíso, Chile*

Resumen

Los algoritmos de inteligencia de enjambres son eficaces en la resolución de problemas de optimización, pero frecuentemente enfrentan limitaciones significativas debido a su convergencia prematura y estancamiento en óptimos locales. Tradicionalmente, esta problemática se aborda mediante el ajuste manual de parámetros, una práctica ineficaz que tiende a comprometer el delicado equilibrio entre la exploración y la explotación. En este contexto, el presente trabajo tiene como objetivo desarrollar un controlador en línea capaz de justificar dinámicamente sus decisiones para evitar dichos estancamientos. A nivel conceptual, la metodología propuesta integra técnicas de explicabilidad para revelar el funcionamiento interno de estos algoritmos, transformando su inherente comportamiento de caja negra en un sistema operativo transparente. El aporte principal de esta investigación radica en la formulación de un procedimiento claro, aplicable y con un alto potencial de replicación, que permite una adaptación paramétrica fundamentada durante la ejecución. Finalmente, la validación del controlador se realiza empíricamente aplicándolo a escenarios de logística de última milla establecidos en la literatura, comprobando así la viabilidad y la mejora en el rendimiento del enfoque propuesto.

Palabras clave: Inteligencia de enjambres, Metaheurísticas bio-inspiradas, SeaHorse Optimization Algorithm, Inteligencia Artificial Explicable, LIME, Controlador en línea, Estancamiento, Convergencia prematura

Metadata del software

Tabla 1: Información del código y repositorio

Descripción	Metadata
Versión actual del código	1.0
Email de soporte	maximiliano.aguirre@estudiantes.uv.cl

Tabla de especificaciones del método

*Contacto: Maximiliano Alejandro Aguirre Faúndes (maximiliano.aguirre@estudiantes.uv.cl)

Tabla 2: Información general del método

Descripción	Detalle
Área temática	[Seleccionar área]
...	...
Disponibilidad de recursos	[Links a datos, software, hardware, etc.]

1. Motivación y contexto

La resolución de problemas de optimización de alta complejidad, frecuentemente clasificados como NP-Hard, resulta inviable mediante modelamiento matemático exacto debido al crecimiento exponencial de su espacio de soluciones a medida que aumentan las variables [1]. Frente a este inmenso desafío, los algoritmos bioinspirados basados en inteligencia de enjambres, como el SeaHorse Optimization Algorithm (SHO), han emergido como herramientas de búsqueda altamente eficientes [2]. Estas metaheurísticas resultan vitales en aplicaciones prácticas del mundo real, tales como la planificación de logística de última milla y la calendarización industrial, permitiendo explorar vastos espacios para obtener soluciones factibles de alta calidad en tiempos computacionales razonables [3, 4].

A pesar de su notable eficacia empírica, la implementación de estas metaheurísticas enfrenta un desafío crítico y persistente: la susceptibilidad a sufrir una convergencia prematura y estancarse en óptimos locales [5]. Este fenómeno ocurre cuando el algoritmo pierde su diversidad poblacional rápidamente y concentra su búsqueda en regiones subóptimas, fracasando en la identificación del óptimo global. La raíz de este problema radica en la naturaleza de caja negra intrínseca a estos modelos estocásticos. Al operar con hiperparámetros predefinidos y depender de transiciones basadas en la aleatoriedad, las complejas dinámicas internas que conducen al fracaso o éxito permanecen completamente ocultas durante toda la ejecución [6].

Tradicionalmente, la comunidad científica ha intentado mitigar estas deficiencias mediante el ajuste manual de parámetros basado en el criterio de experto. Sin embargo, esta práctica resulta ineficaz, costosa y poco robusta. Un parámetro óptimo para una fase exploratoria inicial puede volverse perjudicial en etapas avanzadas, rompiendo el delicado equilibrio requerido entre diversificación e intensificación [7]. Aunque enfoques metodológicos recientes emplean hiperheurísticas para automatizar este control, la gran mayoría opera bajo lógicas opacas sin ofrecer transparencia sobre sus decisiones [8]. Adicionalmente, las técnicas de Inteligencia Artificial Explicable (XAI) aplicadas a este dominio se han limitado casi enteramente a diagnósticos post-ejecución; operan como auditorías que explican el comportamiento únicamente una vez finalizada la búsqueda, siendo incapaces de intervenir, justificar y corregir el rumbo dinámicamente en tiempo real [9, 6].

En esta profunda brecha metodológica se fundamenta el presente trabajo, ante la urgencia de evolucionar desde optimizadores ciegos hacia sistemas de búsqueda adaptativos y transparentes. Este problema central motiva el desarrollo de un método consistente en la integración de un controlador en línea dentro de la metaheurística base. Este controlador emplea técnicas de explicabilidad, específicamente el modelo agnóstico LIME, operando de forma activa in-ejecución [10]. El método propuesto monitorea continuamente métricas de diagnóstico vitales del enjambre, como la densidad poblacional y la tasa de mejora inter-iteracional, para diagnosticar el riesgo inminente de estancamiento. Al generar explicaciones locales en cada iteración, el método justifica matemáticamente las intervenciones paramétricas dinámicas, evitando activamente que el algoritmo quede atrapado.

La relevancia de esta propuesta radica en proveer trazabilidad e interpretabilidad a procesos estocásticos complejos, transformando un paradigma heurístico de ensayo y error en un procedimiento guiado lógicamente. Este método resulta indispensable en cualquier dominio industrial, particularmente en la logística de última milla, donde el estancamiento se traduce en altos costos operativos o ineficiencias críticas [11, 4]. Dotar a las metaheurísticas de explicabilidad activa no solo optimiza su rendimiento y robustez, sino que aporta la transparencia metodológica esencial para su adopción confiable en la industria.

2. Descripción del método

El método propuesto en esta investigación consiste en el desarrollo e integración de un controlador en línea dotado de explicabilidad activa, diseñado específicamente para guiar algoritmos de optimización bioinspirados. Su objetivo principal es anticipar y mitigar problemas críticos inherentes a estas metaheurísticas, tales como la convergencia prematura y el estancamiento en óptimos locales. Con esta implementación, se busca transformar la metaheurística de un sistema estático y opaco, tradicionalmente considerado como una 'caja negra', en un modelo dinámico, transparente y autoadaptable. A diferencia de los enfoques convencionales que dependen casi exclusivamente del ajuste manual de parámetros basado en el ensayo y error o en el criterio de experto, este método automatiza la toma de decisiones direccionales basándose en la justificación analítica de su comportamiento durante el tiempo de ejecución.

A nivel conceptual, el método opera mediante el monitoreo orientado a eventos de diagnóstico internos del enjambre, tales como la tasa de mejora inter-iteracional. La característica más distintiva y el mayor aporte de esta propuesta radica en la incorporación intrínseca de un módulo de Inteligencia Artificial Explicable (XAI), utilizando específicamente el modelo agnóstico LI-ME. En lugar de limitar el uso de la explicabilidad a una mera auditoría forense post-ejecución, el controlador emplea las explicaciones generadas en tiempo real para diagnosticar el riesgo inminente de estancamiento en la búsqueda.

Una vez detectado este riesgo operativo, el sistema interviene la ejecución de manera fundamentada, ajustando dinámicamente los hiperparámetros del algoritmo para forzar transiciones estratégicas y oportunas entre las fases de exploración y explotación. En consecuencia, el aporte fundamental del método es dotar a la búsqueda estocástica de una trazabilidad absoluta y una profunda coherencia operativa; cada decisión de ajuste está respaldada por una explicación interpretable. La implementación de este controlador no solo persigue mejorar sustancialmente el rendimiento y la robustez del algoritmo frente a problemas de alta complejidad combinatoria, sino que aporta la transparencia metodológica indispensable para su adopción confiable en escenarios reales de la industria, como la planificación logística.

3. Fundamentos del método

El método propuesto se fundamenta en la integración sinérgica de tres pilares teóricos fundamentales: la optimización mediante inteligencia de enjambres, el análisis dinámico de sistemas estocásticos y la Inteligencia Artificial Explicable (XAI) [3, 6]. A continuación, se detallan los principios, modelos y supuestos que sustentan la arquitectura del controlador en línea diseñado.

Modelado de la Metaheurística Base, el sistema utiliza como núcleo de búsqueda el algoritmo de optimización bioinspirado *SeaHorse Optimization* (SHO). Este modelo matemático emula el comportamiento social, de forrajeo y reproductivo de los caballitos de mar [2]. Desde una perspectiva teórica, el enjambre se modela como una población de agentes distribuidos en un espacio de búsqueda D-dimensional, donde cada agente representa una solución candidata al problema de optimización.

El principio operativo de SHO asume que la convergencia hacia el óptimo global se logra mediante la alternancia iterativa de tres comportamientos fundamentales:

- **Movimiento (Exploración):** Impulsado por movimientos de tipo espiral o trayectorias aleatorias, permitiendo a los agentes mapear áreas inexploradas del espacio de búsqueda.

- **Depredación (Explotación):** Modelado matemáticamente para intensificar la búsqueda en la vecindad de las mejores soluciones encontradas hasta el momento, simulando la captura de presas.
- **Reproducción:** Un mecanismo de herencia o cruce que busca transmitir características genéticas favorables a la siguiente generación de agentes, manteniendo un nivel de variabilidad.

El supuesto central en el diseño estándar de SHO (y de la gran mayoría de las metaheurísticas poblacionales) es que el equilibrio entre exploración y explotación puede regirse por una regla de transición estocástica simple [7]. Típicamente, el algoritmo utiliza una variable aleatoria $r_1 \sim \mathcal{U}(0, 1)$ evaluada en cada iteración; si $r > 0,5$, el enjambre tiende a explotar, de lo contrario, explora. Sin embargo, este supuesto es metodológicamente deficiente frente a paisajes de búsqueda multimodales complejos, ya que la transición es 'ciega' a la topología del problema y al estado actual del enjambre [5].

El método propuesto parte del principio de que el estancamiento en óptimos locales no es un evento repentino, sino un proceso degenerativo que puede ser diagnosticado temporalmente [5]. El estancamiento se conceptualiza como la condición en la cual la tasa de mejora de la función objetivo (fitness) se aproxima a cero durante una ventana iterativa determinada, mientras que la población converge espacialmente en una región subóptima.

Para diagnosticar este estado, el método asume la necesidad de medir métricas del enjambre en tiempo real. Los conceptos de diagnóstico incluyen: Para diagnosticar de manera precisa este estado operativo, el método no se basa en un simple umbral estático de tolerancia, sino que implementa un sistema de validación multicondicional a través del controlador en línea. Las métricas y condiciones de diagnóstico incluyen:

- **Varianza de Ventana Temporal:** El sistema monitorea la desviación estándar del *fitness* de las mejores soluciones durante una ventana de iteraciones predefinida. Si esta varianza métrica cae por debajo de un umbral crítico ($\epsilon_{stagnation}$), se activa un primer disparador lógico (*trigger*) que marca a la población como candidata a sufrir estancamiento.
- **Mejora Esperada Simulada mediante LIME (Δf):** Una vez que la varianza invoca al módulo XAI, LIME genera una predicción del comportamiento local del enjambre. Se estima un diferencial esperado de mejora definido matemáticamente como $\Delta f = f_{actual} - f_{candidato}$
- **Diagnóstico de Estancamiento Positivo (*Positive Stagnation*):** Para que el controlador justifique y declare formalmente un estado de estancamiento inminente, no basta con observar el diferencial Δf . El método exige que se cumplan simultáneamente tres condiciones derivadas de la explicación de LIME:
 1. Una fuerte importancia estocástica detectada en la regla de transición de la metaheurística.
 2. Una mejora esperada catalogada como baja o nula ($\Delta f < \delta_{tolerance}$).
 3. Un nivel de fidelidad del modelo subrogado (*fidelity threshold*) lo suficientemente alto como para confiar estadísticamente en la predicción de LIME.
- **Evaluación Post-Rescate y Retroceso (*Rollback*):** Tras justificar y aplicar una intervención de rescate paramétrico, el controlador ejecuta una evaluación empírica real en las

siguientes iteraciones. Si, tras un periodo de paciencia algorítmica, la mejora no resulta suficiente en comparación al costo computacional del salto exploratorio, el sistema posee la capacidad de revertir los cambios de estado (*rollback*), salvaguardando así la estabilidad y convergencia de la búsqueda.

El principio de explicabilidad local, modelo LIME, el corazón del controlador en línea radica en la aplicación de técnicas de interpretabilidad para justificar las intervenciones paramétricas [9]. Para ello, el método se basa en el modelo *Local Interpretable Model-agnostic Explanations* (LIME) [12, 13].

El principio teórico de LIME es que, aunque el comportamiento global de un modelo de caja negra" (como la dinámica completa del enjambre a lo largo de todas las iteraciones) es demasiado complejo para ser interpretado directamente, su comportamiento *local* alrededor de una iteración específica puede ser aproximado con alta fidelidad mediante un modelo interpretable más simple, típicamente una regresión lineal penalizada.

Formalmente, dada una iteración crítica donde se detecta un posible estancamiento, el controlador perturba el espacio de características del enjambre (variando sintéticamente parámetros como la diversidad o el peso de inercia) y evalúa el impacto de estas perturbaciones en una función objetivo subrogada (la probabilidad de estancamiento). LIME resuelve el siguiente problema de optimización local:

$$\xi(x) = \arg \min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \quad (1)$$

Donde f es el modelo subyacente complejo, g es el modelo explicativo lineal, π es una medida de proximidad (que da más peso a los estados del enjambre más cercanos a la iteración actual), y $\Omega(g)$ penaliza la complejidad de la explicación para mantenerla interpretable.

Bajo estos conceptos, el método propuesto unifica ambos mundos. Se asume que el comportamiento de la metaheurística es modificable en tiempo de ejecución alterando sus hiperparámetros directrices. Las características extraídas del enjambre alimentan el controlador, y LIME proporciona una asignación de pesos a dichas características, explicando matemáticamente qué factor está causando el deterioro del rendimiento [10].

Por ejemplo, si la explicación de LIME indica que la variable "*Baja Diversidad*" tiene el peso predictivo dominante hacia el estancamiento, el controlador ejecuta una intervención informada: anula temporalmente la regla de transición estocástica (r) de SHO y modifica el factor de depredación o el peso de inercia para inyectar un comportamiento de exploración forzada.

En resumen, el método se fundamenta en el principio de que la optimización metaheurística no debe basarse en un empirismo probabilístico ciego, sino en un control dinámico de retroalimentación donde la interpretabilidad matemática justifica, en determinados momentos, la calibración adaptativa del algoritmo, permitiendo su aplicación robusta y coherente [6].

4. Detalles de implementación

Entorno de desarrollo y librerías, la implementación se realizó en Python 3.14 dentro de un entorno virtual aislado, con instalación de dependencias vía archivo de requisitos. Las librerías principales fueron: NumPy (cálculo numérico), SciPy (estadística), pandas (procesamiento tabular), matplotlib (visualización), opfunu (funciones CEC 2022), lime (explicabilidad local) y scikit-learn (dependencia de LIME) finalmente, para asegurar compatibilidad con opfunu se fijó además setuptools en versión 75.8.0.

Descripción general del pipeline; el procedimiento completo se estructuró en cuatro fases:

1. Preparación del entorno y dependencias.
2. Ejecución de benchmarks para:
 - SHOA + LIME (método propuesto),
 - SHOA puro (línea base),
 - PSO (línea base en escenario validado).
3. Registro exhaustivo de trazas por iteración y por corrida.
4. Postproceso estadístico y visual:
 - métricas Best/Worst/Mean/Std de fitness final,
 - curvas de convergencia por función/instancia,
 - comparación consolidada entre métodos,
 - pruebas de normalidad y contraste no paramétrico.

Para evaluar rigurosamente la eficacia, robustez y capacidad adaptativa del controlador en línea basado en explicabilidad (XAI), el diseño experimental se estructuró en dos escenarios de validación complementarios: un entorno de matemático estándar y un problema aplicado de alta complejidad combinatoria.

En una primera fase de experimentación puramente matemática, se utilizó el conjunto de problemas del *Congress on Evolutionary Computation* (CEC 2022). Específicamente, se evaluó la suite completa de doce funciones ($F1 - F12$). Para garantizar la estandarización y reproducibilidad de las pruebas, el cálculo de la función objetivo se integró mediante llamadas directas a la librería `opf.unu`. Todos los experimentos reportados bajo este entorno se ejecutaron fijando una dimensionalidad de $d = 10$, con los límites del espacio de búsqueda estrictamente acotados en el intervalo $[-100, 100]$, tal como lo define el estándar de la competencia.

En una segunda fase, orientada a la validación empírica en un contexto industrial, se abordó el **Problema de Localización y Asignación de Microhubs Temporales** (TMLAP, por sus siglas en inglés). Este es un modelo de optimización combinatoria enfocado en la logística de última milla, cuyo objetivo principal radica en minimizar los costos sistémicos mediante la selección óptima de puntos de retiro temporales a habilitar y la asignación eficiente de clientes a estos, sujeto a restricciones estrictas de capacidad operativa y umbrales máximos de desplazamiento.

La justificación para emplear este escenario en la presente investigación —originado a partir de un ejercicio propuesto por el profesor guía— radica en que representa un caso de estudio realista de los desafíos logísticos urbanos modernos, adaptado a complejidades geográficas reales como las de la ciudad de Valparaíso. Al constituir un problema NP-Hard con variables de decisión discretas y de alta exigencia algorítmica, el TMLAP ofrece un entorno de prueba riguroso que permite demostrar el valor práctico del controlador en la resolución de problemas industriales complejos, yendo más allá de las funciones continuas de la CEC 2022.

Finalmente, dado que la metaheurística base opera en un espacio continuo, la resolución del TMLAP requirió una adaptación en la codificación de los individuos. Para ello, se empleó una representación latente continua $z \in \mathbb{R}^{n_{\text{clientes}}}$, la cual es procesada en cada iteración mediante una función de reparación algorítmica encargada de decodificarla y transformarla en una asignación discreta y factible de clientes a microhubs. Este dicho reparador aplica:

1. cálculo de hubs factibles por cliente bajo restricción de distancia $D_{\max}$,

2. ordenamiento de clientes por nivel de restricción (menos opciones primero),
3. asignación al hub factible más cercano al valor latente, respetando capacidades,
4. verificación final de factibilidad.

La función objetivo minimiza costo total de asignación + apertura de hubs. Si una solución no puede repararse, se asigna penalización alta (10^9).

La arquitectura del sistema propuesto se divide en dos componentes fundamentales: la implementación estricta del algoritmo metaheurístico base y el diseño del controlador autónomo basado en explicabilidad.

Algoritmo Base: SeaHorse Optimization, el optimizador central del sistema es el algoritmo bioinspirado SHO, el cual se implementó fielmente en tres etapas secuenciales por cada iteración: movimiento, depredación y reproducción-selección.

Movimiento: En esta fase exploratoria, la transición espacial de los individuos se rige por un vector de decisión estocástico que determina la naturaleza del salto. Este vector se compone de cinco variables clave:

$$(r_1, \text{mag}_{brown}, \text{mag}_{levy}, r_2, \text{mag}_{pred}).$$

Estas variables permiten al enjambre alternar probabilísticamente entre movimientos brownianos y vuelos de Lévy para explorar el hiperespacio de soluciones.

Depredación: Esta fase intensifica la búsqueda local, acercando a las soluciones candidatas hacia las regiones más prometedoras. Para moderar la agresividad de este acercamiento a lo largo del tiempo, se aplica un factor de escala dinámico:

$$\alpha_t = (1 - t/T)^{2t/T+\epsilon},$$

donde t representa la iteración actual y T el presupuesto máximo de iteraciones, logrando un balance progresivo entre exploración y explotación.

Reproducción y Selección: Para mantener la diversidad y transferir información valiosa, la población candidata se ordena según su *fitness*. El cruce se realiza combinando a la mitad superior de la población (considerados como "padres") con la mitad restante ("madres") mediante la ecuación de herencia:

$$x_{off} = r_3 x_{father} + (1 - r_3) x_{mother}.$$

Finalmente, para mantener el tamaño poblacional constante, se evalúa el *fitness* de los nuevos individuos y se seleccionan las mejores N soluciones combinando la población candidata y su descendencia.

Controlador SHO + LIME (Método Propuesto), la principal contribución metodológica de este trabajo radica en la incorporación de un controlador en línea sobre el algoritmo SHO. Este módulo proporciona diagnóstico explicable y capacidades de rescate autónomo para evitar el estancamiento. Su ciclo operativo se compone de cuatro etapas.

Detección de Estancamiento El controlador monitorea continuamente la salud del proceso de búsqueda mediante una ventana deslizante que almacena el mejor *fitness* histórico. El algoritmo se marca como candidato a recibir un diagnóstico formal si, una vez completada la ventana temporal, la varianza del *fitness* cae por debajo de un umbral crítico:

$$\text{std}(\text{ventana}) < \epsilon_{stagnation}$$

Diagnóstico Local con LIME Al activarse la bandera de estancamiento, el módulo de Inteligencia Artificial Explicable (XAI) interviene. Se utiliza el modelo LIME para aproximar localmente el comportamiento del enjambre, evaluando qué factores algorítmicos indujeron el fallo en la convergencia. Para ello, se toma como conjunto de características explicativas el vector estocástico que rige el desplazamiento de los individuos:

$$\{r_1, \text{mag}_{\text{browniano}}, \text{mag}_{\text{levy}}, r_2, \text{mag}_{\text{predacion}}\}.$$

En este vector, las variables de magnitud (mag) representan la intensidad o el tamaño del paso calculado para cada comportamiento simulado en la iteración. Específicamente, $\text{mag}_{\text{browniano}}$ cuantifica la amplitud de la trayectoria de exploración local basada en el movimiento browniano; mag_{levy} define la escala espacial de los saltos largos y esporádicos característicos de los vuelos de Lévy (cruciales para escapar de óptimos locales); y $\text{mag}_{\text{predacion}}$ mide la fuerza con la que el agente es atraído hacia la solución élite (o repelido de ella) durante la fase de explotación.

Al analizar la influencia combinada de estos tamaños de paso junto con las probabilidades de transición (r_1 y r_2), LIME genera un modelo subrogado local. Este modelo permite predecir la mejora esperada de la función objetivo, definida matemáticamente como $\Delta f = f_{\text{actual}} - f_{\text{candidato}}$. A esta estimación, que actúa como la justificación analítica del módulo explicable sobre el potencial real de la iteración, se le denomina *pred_delta*.

Regla de Estancamiento Positivo Para evitar intervenciones innecesarias, el controlador no actúa ante cualquier desaceleración del algoritmo. Se declara el estado de *POSITIVE_STAGNATION* única y exclusivamente si se cumplen de forma simultánea las siguientes tres condiciones derivadas del diagnóstico de LIME:

1. **Importancia estocástica fuerte:** Las magnitudes explicativas asignadas por LIME a los términos estocásticos superan el umbral de importancia establecido.
2. **Baja mejora esperada:** El diferencial pronosticado es inferior a la tolerancia permitida ($\text{pred_delta} < \delta_{\text{tol}}$).
3. **Fidelidad del modelo local:** El ajuste del modelo subrogado es estadísticamente aceptable ($\text{fidelity} \geq \tau_{\text{fid}}$) o, en su defecto, no finito, lo que autoriza la toma de decisiones.

Rescate y Rollback Confirmado el estancamiento positivo, el controlador interviene el flujo normal de la metaheurística aplicando un rescate paramétrico forzado. El objetivo central de esta fase es inyectar diversidad poblacional de manera abrupta para romper la atracción gravitacional de un óptimo local profundo. Para lograr este cometido, el sistema activa una de las siguientes estrategias de mutación o perturbación espacial, dependiendo de cual se configuró anteriormente:

- **Repulsión del Líder (leader_repulsion):** Esta estrategia de mutación direccional se fundamenta en la premisa de que el enjambre ha colapsado prematuramente alrededor de la solución élite actual (G_{best}). Para contrarrestar esta aglomeración, se aplica un vector de fuerza repulsiva. Matemáticamente, se induce un desplazamiento a los agentes en dirección opuesta a la posición de G_{best} , escalado por un factor de intensidad o agresividad (η_{rescue}). Esto obliga a las partículas a alejarse del centro de masa del óptimo local para explorar los límites exteriores de la región, deshaciendo la convergencia excesiva sin perder por completo la información de la vecindad.

- **Teletransportación de Lévy** (*levy_teleport*): Representa un mecanismo de escape exploratorio mucho más radical, basado en la distribución probabilística de cola pesada característica de los vuelos de Lévy. Al activarse, esta mutación reubica a una fracción de los individuos aplicando saltos estocásticos de gran magnitud. A diferencia del ruido gaussiano estándar, la distribución de Lévy genera ocasionalmente desplazamientos extremos, "teletransportando" a los agentes a regiones del hiperespacio de soluciones completamente inexploradas. Esta estrategia es vital cuando el enjambre se encuentra atrapado en valles de estancamiento sumamente amplios, donde un escape local no sería suficiente.

Tras la inyección de estas mutaciones disruptivas, el algoritmo entra en un periodo de gracia transitorio denominado 'paciencia algorítmica' (*rescue_patience_iters*). Durante este lapso iterativo continuo, el enjambre tiene la oportunidad de asimilar la nueva distribución espacial y reanudar su dinámica de convergencia natural.

Si al finalizar este periodo de evaluación, la mejora empírica del *fitness* global no logra superar el umbral de aceptación mínimo predefinido (*rescue_min_improvement*), el sistema concluye que la exploración forzada fue infructuosa. Ante este escenario, el controlador ejecuta un mecanismo de recuperación de estado o *rollback*. Esta función restaura de manera exacta las posiciones, velocidades e hiperparámetros del enjambre al estado seguro registrado justo en la iteración previa al rescate. Este mecanismo de protección es esencial para dotar al controlador de robustez, evitando la dispersión destructiva y la degradación acumulada que provocarían saltos exploratorios ineficaces.

Configuración Experimental

Parámetros Base (Benchmark Principal) Para garantizar la reproducibilidad, la configuración base del controlador se detalla en la Tabla 3.

Parámetro	Valor
Ejecuciones independientes (<i>runs</i>)	30
Semilla inicial (<i>seed_start</i>)	1
Tamaño poblacional (<i>pop_size</i>)	30
Iteraciones máximas (<i>max_iter</i>)	500
Tamaño de ventana LIME (<i>window_size</i>)	10
Umbral estancamiento ($\epsilon_{stagnation}$)	8×10^{-4}
Enfriamiento post-rescate (<i>cooldown</i>)	6
Muestras LIME (<i>lime_samples</i>)	1200
Umbral de importancia (<i>importance_threshold</i>)	0.035
Tolerancia de Delta (δ_{tol})	8×10^{-5}
Fidelidad mínima (τ_{fid})	0.10
Modo de rescate (<i>rescue_mode</i>)	<i>leader_repulsion</i>
Eta de rescate (<i>rescue_eta</i>)	1.0
Escala Lévy para rescate (<i>rescue_levy_scale</i>)	0.30
Paciencia de rescate (<i>rescue_patience_iters</i>)	25
Mejora mínima de rescate (<i>rescue_min_improvement</i>)	0.0
Forzar archivo elite (<i>enforce_elite_archive</i>)	True

Tabla 3: Configuración paramétrica base del controlador SHO + LIME.

Perfiles de Agresividad y Líneas Base Para el análisis de sensibilidad del controlador, se

definieron tres perfiles de agresividad: *Soft* (activación conservadora y alta paciencia), *Medium* (configuración base descrita en la Tabla 3), y *Hard* (activación rápida y rescates agresivos).

El rendimiento del controlador se comparó contra dos algoritmos de referencia sin capacidades dinámicas de diagnóstico:

- **SHOA original:** Mismo protocolo de población e iteraciones, pero carente del diagnóstico y rescate proporcionado por LIME.
- **PSO estándar (aplicado al problema TMLAP):** Configurado con inercia $w = 0,72$, coeficientes de aceleración $c_1 = c_2 = 1,70$, y un límite de velocidad fraccional de $v_{max} = 0,25$.

Trazabilidad y Postproceso Estadístico Para asegurar la auditoría de los experimentos, el sistema genera trazas por iteración, las contribuciones locales de LIME, rankings, un archivo JSON de configuración y registros de fallos.

El análisis de los resultados consolidados contempló la extracción de métricas descriptivas (*best*, *worst*, *mean*, *std*), la generación de curvas de convergencia con bandas de variabilidad, y una validación estadística formal. El protocolo estadístico requirió verificar la normalidad de los datos mediante el test de Shapiro-Wilk, seguido de la aplicación de la prueba no paramétrica de Wilcoxon para muestras pareadas, estableciendo un nivel de significancia de $\alpha = 0,05$. Bajo este criterio, un valor $p < 0,05$ determina una diferencia estadísticamente significativa a favor del enfoque propuesto.

5. Validación del método

Para demostrar la viabilidad, robustez y aplicabilidad del controlador en línea basado en explicabilidad, el método propuesto fue sometido a un riguroso protocolo de validación [3]. Este proceso experimental se dividió en dos entornos de prueba complementarios: un entorno matemático estandarizado y un escenario logístico aplicado de alta complejidad [4].

El primer entorno de prueba utilizó la suite de funciones del *Congress on Evolutionary Computation* (CEC 2022) [14]. Se emplearon las 12 funciones *benchmark* ($F1 - F12$) diseñadas para evaluar rigurosamente algoritmos de optimización con restricciones de límite. Las pruebas se configuraron en 10 dimensiones ($d = 10$) haciendo uso de la librería *opfunu*. Para garantizar la validez estadística y capturar la variabilidad estocástica inherente a la metaheurística, se ejecutaron 30 corridas independientes por cada función, con un límite estricto de 500 iteraciones máximas. Este escenario permitió aislar y evaluar el comportamiento matemático del algoritmo frente a topologías de búsqueda puramente teóricas y desafiantes.

El segundo entorno de validación consistió en el Problema de Localización y Asignación de Microhubs Temporales (TMLAP). Este representa un caso de estudio real enfocado en la logística de última milla, modelado específicamente bajo las complejidades topológicas y geográficas de la ciudad de Valparaíso. Al ser un problema *NP-Hard* con restricciones estrictas de capacidad operativa y distancias máximas toleradas, el TMLAP permitió validar el algoritmo propuesto en un espacio de decisiones de alta exigencia industrial. En este escenario, el desempeño del controlador se contrastó tanto con el algoritmo *SeaHorse Optimization* (SHO) base [2] como con una implementación estándar de Optimización por Enjambre de Partículas (PSO) [15].

El módulo de Inteligencia Artificial Explicable (LIME) demostró ser capaz de diagnosticar con precisión el riesgo inminente de estancamiento [6]. De manera autónoma, el sistema logró

gatillar las estrategias de rescate paramétrico (`leader_repulsion` y `levy_teleport`) únicamente cuando la mejora local pronosticada era insuficiente [10].

— aquí añadiré las instancias gigantes —

La evaluación estadística de estos resultados se realizó mediante la prueba no paramétrica de Wilcoxon [16], aplicada tras confirmar la no normalidad de las distribuciones con el test de Shapiro-Wilk [17]. El contraste formal confirmó que el método propuesto alcanza mejoras significativas ($p < 0,05$) en la calidad global de las soluciones encontradas. Esta evidencia confirma de manera concluyente que la integración de la explicabilidad activa logra mitigar la convergencia prematura, demostrando que el sistema es plenamente aplicable y superior tanto en paisajes matemáticos complejos como en escenarios operativos reales.

6. Limitaciones

Las limitaciones del método propuesto están intrínsecamente ligadas a la naturaleza estocástica de las metaheurísticas y a la sobrecarga analítica del módulo de explicabilidad. En primer lugar, debido a su condición heurística, el algoritmo no garantiza matemáticamente la convergencia absoluta hacia el óptimo global, sino que asegura el hallazgo de soluciones factibles de alta calidad. Desde una perspectiva técnica, la efectividad del rescate autónomo depende estrictamente de la fidelidad del modelo subrogado generado por LIME; si la aproximación lineal local no logra capturar adecuadamente la topología subyacente en hiperespacios de búsqueda altamente no lineales, el diagnóstico de estancamiento podría resultar impreciso. Adicionalmente, el método presenta una restricción de escalabilidad computacional: la necesidad de generar perturbaciones sintéticas y evaluar la mejora esperada durante las iteraciones críticas introduce un costo temporal extra. Esto podría limitar la aplicabilidad del controlador en escenarios de optimización que requieran resoluciones estrictamente en tiempo real o en problemas con una dimensionalidad excesivamente alta donde el muestreo local pierda representatividad (maldición de la dimensionalidad).

Agradecimientos

Los autores agradecen a la Escuela de Ingeniería Informática de la Universidad de Valparaíso.

Aspectos éticos

Confirmamos que el trabajo...

Declaración de autoría

Autor 1: Conceptualización, ... Autor 2: Software, ...

Declaración de intereses

Los autores declaran que no hay conflictos de intereses. Si hay conflictos de intereses, detallar acá.

Referencias

- [1] S. K. Jena, K. Subramani, A. Velasquez, A Differential Approach for Several NP-hard Optimization Problems, Springer Nature Switzerland, 2024, p. 68–80. doi:10.1007/978-3-031-63735-3_4.
URL http://dx.doi.org/10.1007/978-3-031-63735-3_4
- [2] S. Zhao, T. Zhang, S. Ma, M. Wang, Sea-horse optimizer: a novel nature-inspired meta-heuristic for global optimization problems, Applied Intelligence 53 (10) (2022) 11833–11860. doi:10.1007/s10489-022-03994-3.
URL <http://dx.doi.org/10.1007/s10489-022-03994-3>
- [3] E. Osaba, E. Villar-Rodriguez, J. Del Ser, A. J. Nebro, D. Molina, A. LaTorre, P. N. Suganthan, C. A. Coello Coello, F. Herrera, A tutorial on the design, experimentation and application of metaheuristic algorithms to real-world optimization problems, Swarm and Evolutionary Computation 64 (2021) 100888. doi:10.1016/j.swevo.2021.100888.
URL <http://dx.doi.org/10.1016/j.swevo.2021.100888>
- [4] E. Rodríguez-Esparza, A. D. Masegosa, D. Oliva, E. Onieva, A new hyper-heuristic based on adaptive simulated annealing and reinforcement learning for the capacitated electric vehicle routing problem, Expert Systems with Applications 252 (2024) 124197. doi:10.1016/j.eswa.2024.124197.
URL <http://dx.doi.org/10.1016/j.eswa.2024.124197>
- [5] M. Črepinšek, M. Mernik, M. Beković, M. Pintarič, M. Moravec, M. Ravber, Overcoming stagnation in metaheuristic algorithms with msma’s adaptive meta-level partitioning, Mathematics 13 (11) (2025) 1803. doi:10.3390/math13111803.
URL <http://dx.doi.org/10.3390/math13111803>
- [6] J. Almeida, J. Soares, F. Lezama, S. Limmer, T. Rodemann, Z. Vale, A systematic review of explainability in computational intelligence for optimization, Computer Science Review 57 (2025) 100764. doi:10.1016/j.cosrev.2025.100764.
URL <http://dx.doi.org/10.1016/j.cosrev.2025.100764>
- [7] Y. Zhang, G. Chen, L. Cheng, Q. Wang, Q. Li, Methods to balance the exploration and exploitation in differential evolution from different scales: A survey, Neurocomputing 561 (2023) 126899. doi:10.1016/j.neucom.2023.126899.
URL <http://dx.doi.org/10.1016/j.neucom.2023.126899>
- [8] T. Dokeroğlu, T. Kucukyilmaz, E.-G. Talbi, Hyper-heuristics: A survey and taxonomy, Computers & Industrial Engineering 187 (2024) 109815. doi:10.1016/j.cie.2023.109815.
URL <http://dx.doi.org/10.1016/j.cie.2023.109815>
- [9] B. H. Abed-Alguni, Evomapx: An explainable framework for metaheuristic optimization algorithms, Expert Systems with Applications 298 (2026) 129514. doi:10.1016/j.eswa.2025.129514.
URL <http://dx.doi.org/10.1016/j.eswa.2025.129514>

- [10] H. Nakagawa, S. Sumita, R. Iida, T. Tsuchiya, An xai-based meta-parameter tuning for time-series forecasting, *Annals of Mathematics and Artificial Intelligence* (Mar. 2025). doi:10.1007/s10472-025-09973-x.
URL <http://dx.doi.org/10.1007/s10472-025-09973-x>
- [11] T. Barros-Everett, E. Montero, N. Rojas-Morales, Parameter prediction for metaheuristic algorithms solving routing problem instances using machine learning, *Applied Sciences* 15 (6) (2025) 2946. doi:10.3390/app15062946.
URL <http://dx.doi.org/10.3390/app15062946>
- [12] L. Longo, M. Brcic, F. Cabitza, J. Choi, R. Confalonieri, J. D. Ser, R. Guidotti, Y. Hayashi, F. Herrera, A. Holzinger, R. Jiang, H. Khosravi, F. Lecue, G. Malgieri, A. Páez, W. Samek, J. Schneider, T. Speith, S. Stumpf, Explainable artificial intelligence (xai) 2.0: A manifesto of open challenges and interdisciplinary research directions, *Information Fusion* 106 (2024) 102301. doi:10.1016/j.inffus.2024.102301.
URL <http://dx.doi.org/10.1016/j.inffus.2024.102301>
- [13] K. Kalasampath, K. N. Spoorthi, S. Sajeev, S. S. Kuppa, K. Ajay, A. Maruthamuthu, A literature review on applications of explainable artificial intelligence (xai), *IEEE Access* 13 (2025) 41111–41140. doi:10.1109/access.2025.3546681.
URL <http://dx.doi.org/10.1109/ACCESS.2025.3546681>
- [14] A. W. Mohamed, K. M. Sallam, P. Agrawal, A. A. Hadi, A. K. Mohamed, Evaluating the performance of meta-heuristic algorithms on cec 2021 benchmark problems, *Neural Computing and Applications* 35 (2) (2022) 1493–1517. doi:10.1007/s00521-022-07788-z.
URL <http://dx.doi.org/10.1007/s00521-022-07788-z>
- [15] J. Kennedy, R. Eberhart, Particle swarm optimization, in: *Proceedings of ICNN'95 - International Conference on Neural Networks*, Vol. 4 of ICNN-95, IEEE, 1995, p. 1942–1948. doi:10.1109/icnn.1995.488968.
URL <http://dx.doi.org/10.1109/ICNN.1995.488968>
- [16] J. Derrac, S. García, D. Molina, F. Herrera, A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms, *Swarm and Evolutionary Computation* 1 (1) (2011) 3–18. doi:10.1016/j.swevo.2011.02.002.
URL <http://dx.doi.org/10.1016/j.swevo.2011.02.002>
- [17] S. S. Shapiro, M. B. Wilk, An analysis of variance test for normality (complete samples), *Biometrika* 52 (3-4) (1965) 591–611. doi:10.1093/biomet/52.3-4.591.

Anexos

Detalles propios del trabajo. Todo lo que se incluya acá, debe estar referenciado en el documento. La completitud de esta sección es definida por Prof. Guía.

A continuación se detallan anexos sugeridos para la descripción tencológica de los proyectos de desarrollo y de las soluciones informáticas de proyectos de investigación aplicada.

El prof. guía podría sugerir nuevos anexos dependiendo de la naturaleza del trabajo.

Apéndice A. Requerimientos

Apéndice A.1. Modelo de Procesos de Negocio AS-IS

Apéndice A.2. Catálogo de Requerimientos

Apéndice A.3. Historias de Usuario

Apéndice B. Diseño

Apéndice B.1. Arquitectura de Software

Apéndice B.2. Diagrama de Despliegue y Componentes

Apéndice B.3. Diagrama de Componentes y Dependencias

Apéndice B.4. Diagrama de Secuencia

Apéndice B.5. Diagrama de Paquetes

Apéndice B.6. Diagrama de Clases

Apéndice C. Implementación

Apéndice C.1. Stack de desarrollo

Apéndice C.2. Requisitos de hardware para implentación

Apéndice D. Pruebas

Apéndice D.1. Estrategia de Pruebas

Apéndice D.2. Pruebas Funcionales

Apéndice D.3. Pruebas de Integración

Apéndice D.4. Pruebas de Sistema

Apéndice D.5. Pruebas de Aceptación

Apéndice E. Despliege e Implantación

Apéndice E.1. Requisitos de hardware para despliegue

Apéndice E.2. Repositorios de Código

Apéndice E.3. Procedimiento de Despliegue

Apéndice E.4. Procedimientos de Operación